Chapter 4. System Design for Recommending

Now that you have a foundational understanding of how recommendation systems work, let's take a closer look at the elements needed and at designing a system that is capable of serving recommendations at industrial scale. *Industrial scale* in our context will primarily refer to *reasonable scale* (a term introduced by Ciro Greco, Andrea Polonioli, and Jacopo Tagliabue in "ML and MLOps at a Reasonable Scale")—production applications for companies with tens to hundreds of engineers working on the product, not thousands.

In theory, a recommendation system is a collection of math formulas that can take historical data about user-item interactions and return probability estimates for a user-item-pair's affinity. In practice, a recommendation system is 5, 10, or maybe 20 software systems, communicating in real time and working with limited information, restricted item availability, and perpetually out-of-sample behavior, all to ensure that the user sees *something*.

This chapter is heavily influenced by "System Design for Recommendations and Search" by Eugene Yan and "Recommender Systems, Not Just Recommender Models" by Even Oldridge and Karl Byleen-Higley.

Online Versus Offline

ML systems consist of the stuff that you do in advance and the stuff that you do on the fly. This division, between online and offline, is a practical consideration about the information necessary to perform tasks of various types. To observe and learn large-scale patterns, a system needs ac-

cess to lots of data; this is the offline component. Performing inference, however, requires only the trained model and relevant input data. This is why many ML system architectures are structured in this way. You'll frequently encounter the terms *batch* and *real-time* to describe the two sides of the online-offline paradigm (Figure 4-1).

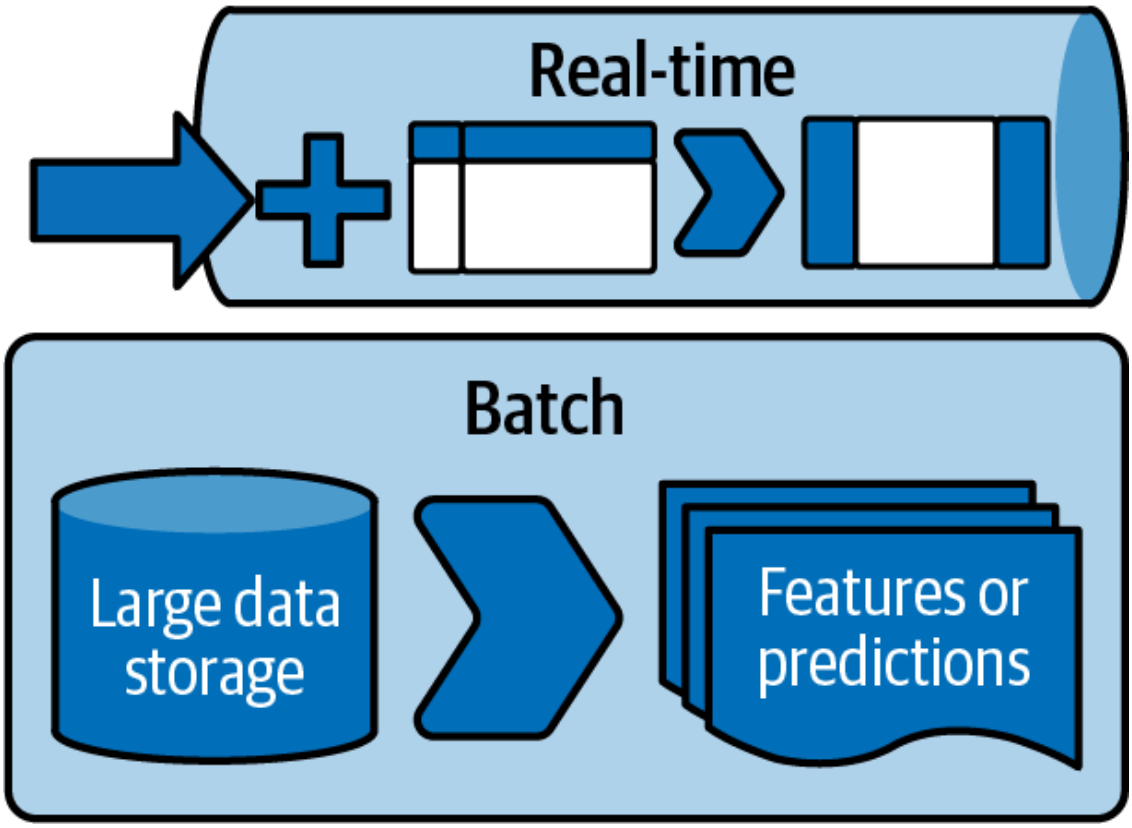

Figure 4-1. Real-time versus batch

A *batch process* does not require user input, often has longer expected time periods for completion, and is able to have all the necessary data available simultaneously. Batch processes often include tasks like training a model on historical data, augmenting one dataset with an additional collection of features, or transforming computationally expensive data. Another characteristic you see more frequently in batch processes is that they work with the full relevant dataset involved, not only an instance of the data sliced by time or otherwise.

A *real-time process* is carried out at the time of the request; said differently, it is evaluated during the inference process. Examples include providing a recommendation upon page load, updating the next episode after the user finishes the last, and re-ranking recommendations after one has been marked *not interesting*. Real-time processes are often resource

constrained because of the need for rapidity, but like many things in this domain, as the world's computational resources expand, we change the definition of resource constrained.

Let's return to the components introduced in Chapter 1—the collector, ranker, and server—and consider their roles in offline and online systems.

Collector

The collector's role is to know what is in the collection of items that may be recommended and the necessary features or attributes of those items.

Offline Collector

The *offline collector* has access to and is responsible for the largest datasets. Understanding all user-item interactions, user similarities, item similarities, feature stores for users and items, and indices for nearest-neighbor lookup are all under the purview of the offline collector. The offline collector needs to be able to access the relevant data extremely fast, and sometimes in large batches. For this purpose, offline collectors often implement sublinear search functions or specifically tuned indexing structures. They may also leverage distributed compute for these transformations.

It's important to remember that the offline collector not only needs access and knowledge of these datasets but will also be responsible for writing the necessary downstream datasets to be used in real time.

Online Collector

The *online collector* uses the information indexed and prepared by the offline collector to provide real-time access to the parts of this data necessary for inference. This includes techniques like searching for nearest neighbors, augmenting an observation with features from a feature store, and knowing the full inventory catalog. The online collector will also

need to handle recent user behavior; this will become especially important when we see sequential recommenders in Chapter 17.

One additional role the online collector may take on is encoding a request. In the context of a search recommender, we want to take the query and encode it into the *search space* via an embedding model. For contextual recommenders, we need to encode the context into the *latent space* via an embedding model also.

EMBEDDING MODELS

One popular subcomponent in the collector's work will involve an embedding step; see *Machine Learning Design Patterns* by Valliappa Lakshmanan et al. (O'Reilly). The embedding step on the offline side involves both training the embedding model and constructing the latent space for later use. On the online side, the embedding transformation will need to embed a query into the right space. In this way, the embedding model serves as a transformation that you include as part of your model architecture.

Ranker

The ranker's role is to take the collection provided by the collector, and order some or all of its elements according to a model for the context and user. The ranker actually gets two components itself, the filtering and the scoring.

Filtering can be thought of as the coarse inclusion and exclusion of items appropriate for recommendation. This process is usually characterized by rapidly cutting away a lot of potential recommendations that we definitely don't wish to show. A trivial example is not recommending items we know the user has already chosen in the past.

Scoring is the more traditional understanding of ranking: creating an ordering of potential recommendations with respect to the chosen objective function.

Offline Ranker

The goal of the *offline ranker* is to facilitate filtering and scoring. What differentiates it from the online ranker is how it runs validation and how the output can be used to build fast data structures that the online ranker can utilize. Additionally, the offline ranker can integrate with a human review process for *human-in-the loop ML*.

An important technology that will be discussed later is the *bloom filter*. A bloom filter allows the offline ranker to do work in batches, so that filtering in real time can happen much faster. An oversimplification of this process would be to use a few features of the request to quickly select subsets of all possible candidates. If this step can be completed quickly—in terms of computational complexity, striving for something less than quadratic in the number of candidates—then downstream complex algorithms can be made much more performant.

Second to the filtering step is the ranking step. In the offline component, ranking is training the model that learns how to rank items. As you will see later, learning to rank items to perform best with respect to the objective function is at the heart of the recommendation models. Training these models, and preparing the aspects of their output, is part of the batch responsibility of the ranker.

Online Ranker

The *online ranker* gets a lot of praise but really utilizes the hard work of other components. The online ranker first does filtering, utilizing the filtering infrastructure built offline—for example, an index lookup or a bloom filter application. After filtering, the number of candidate recommendations has been tamed, and thus we can actually come to the most infamous of the tasks: rank recommendations.

In the online ranking phase, usually a feature store is accessed to take the candidates and embellish them with the necessary details, and then a scoring and ranking model is applied. Scoring or ranking may happen in several independent dimensions and then be collated into one final rank-

ing. In the multiobjective paradigm, you may have several of these ranks associated with the list of candidates returned by a ranker.

Server

The server's role is to take the ordered subset provided by the ranker, ensure that the necessary data schema is satisfied (including essential business logic), and return the requested number of recommendations.

Offline Server

The *offline server* is responsible for high-level alignment of the hard requirements of recommendations returned from the system. In addition to establishing and enforcing schema, these rules can be more nuanced things like "never return this pair of pants when also recommending this top." Often waved off as "business logic," the offline server is responsible for creating efficient ways to impose top-level priorities on the returned recommendations.

An additional responsibility for the offline server is handling tasks like experimentation. At some point, you'll likely want to run online experiments to test out all the amazing recommendation systems you build with this book. The offline server is the place where you'll implement the logic necessary to make experimentation decisions and provide the implications in a way the online server can use them in real time.

Online Server

The *online server* takes the rules, requirements, and configurations established and makes their final application to the ranked recommendations. A simple example is diversification rules; as you will see later, diversification of recommendations can have a significant impact on the quality of a user's experience. The online server can read the diversification requirements from the offline server and apply them to the ranked list to return the expected number of diverse recommendations.

Summary

It's important to remember that the online server is the endpoint from which other systems will be getting a response. While it's usually where the message is coming from, many of the most complicated components in the system are upstream. Be careful to instrument this system in a way that when responses are slow, each system is observable enough that you can identify where those performance degradations are coming from.

Now that we've established the framework and you understand the functions of the core components, we will discuss the aspects of ML systems next and the kinds of technologies associated with them.

In this next chapter, we'll get our hands dirty with the aforementioned components and see how we might implement the key aspects. We'll wrap it up by putting it all together into a production-scale recommender using only the content of each item. Let's go!